Implementation Plan for Corrective Navigation: Ledger, Projection Key, and Audit First

Flyxion
Independent Researcher

October 2026

Abstract

This plan orders the construction of the system specified in *Semantic Geometry and Corrective Navigation* so that every invariant becomes testable as early as possible and no rendering work precedes the layers it depends on. The ledger, the derived graph, and the audit suite come first. A text projection comes next, because it exercises the same graph and the same explanation machinery at a fraction of the cost of a spatial interface. The spatial interface and the experimental testbed come last. Technology choices are stated as proposals with the reasons for them, and the decisions that remain open are listed explicitly. No schedule is given, because capacity is not known; the plan fixes order and dependency only.

1 Governing principles

Three principles determine the order of work. First, the invariants live below the interface, so the layers that carry them are built before any layer that displays them. Second, every invariant is implemented together with its test from the specification, so that a milestone is complete only when its tests pass. Third, ablation switches for the experimental program are designed in from the start, because retrofitting a clean way to violate exactly one invariant is harder than building it alongside the invariant.

2 Proposed architecture

2.1 Components

The system is a pipeline of four components with one-way dependencies.

- **Ledger.** An append-only file of events in a canonical serialization, with each event identified by a hash of its canonical form. This is the only authoritative store.
- **Graph.** A derived index built from the effective events. It is disposable and is rebuilt from the ledger on demand.
- **Projection.** A reproducible layout over the graph, keyed by the projection key from the specification, emitted as a plain data file of positions and encodings.

- **Clients.** A text navigator and a spatial navigator, each reading the projection and the graph and writing back only through ledger events.

2.2 Technology proposals

These are proposals, and each can be changed without affecting the order of work.

Concern	Proposal	Reason
Language for core	Python for the first implementation, with the event schema kept language neutral	Fast iteration while the schema and invariants are still being tested; the schema and test suite carry over if a later rewrite is chosen.
Ledger format	Newline-delimited JSON, canonical key order, SHA-256 event identifiers	Human-readable, diff-friendly under Git, and trivially append-only.
Derived index	SQLite built from the ledger	Fast traversal queries for repair and explanation; safe to delete and rebuild.
Layout	A deterministic layout procedure with an explicit seed and a version identifier	Reproducibility is required by the projection key, so the algorithm must expose its randomness.
Spatial client	Browser based, reading a projection data file	Matches the approach already used in the icepick browser engine, which renders a flattened snapshot as a navigable world.
Tests	One automated test per invariant, run from a single command	The audit arm of the experimental program is this suite.

2.3 Repository layout

```
corrective-nav/
  schema/          event schema, canonical serialization rules
  ledger/          append, validate, hash; no derived state
  graph/           effective-set computation, interpretation, index
  status/          support status, defeat, admissibility derivation
  repair/          dependency traversal, scope, completeness
  projection/      layout procedure, projection key, attribution diff
  explain/         event-chain resolver used by every client
  clients/
    text/          text navigator
    spatial/       browser spatial navigator
  audit/           one test per invariant
  testbed/         planted ground truth, baselines, ablation flags
  corpus/          import scripts and ledger fixtures
```

The directory names are working names. The repository does not exist yet, and its name is an open decision.

3 Milestones

Each milestone lists what it delivers and which invariants become testable as a result. The invariants are numbered as in the specification: (1) ledger primacy, (2) proximity is not authority, (3) channel separation, (4) reproducible projection, (5) retrieval nominates, (6) repair accountability, (7) honest completeness, (8) provenance split, (9) derived admissibility, (10) explanation on demand, (11) unverified marking.

M0. Schema and canonical form

Define the event schema, the canonical serialization, and the identifier rule. Implement validation: a relation has exactly one channel, every event except an assertion has a basis, the kind and relation type belong to the declared sets, and `git_ref` may be null. The deliverable is a validator that rejects malformed events with a stated reason. Testable: the structural half of invariant 3, and invariant 8 in its representability form.

M1. Ledger and derived graph

Implement the append-only ledger, the effective-set computation, and the interpretation into a graph index. The key property is determinism: deleting the index and rebuilding it from the ledger yields an identical graph. Testable: invariant 1.

M2. Status and actors

Implement the status values and the actor model, including retriever processes recorded with model identifier and version. Provisional events must be excluded from any established view by default. Testable: invariants 5 and 11. At the end of this milestone the retriever is a separable component whose removal leaves the established graph unchanged.

M3. Support, defeat, and admissibility

Implement the discrete support status computed from support and defeat events, the relevance relation relative to a context, and the derivation of admissibility from `admit` events and current support. Door state is a function and not a stored field. Testable: invariant 9, and the cross-channel half of invariant 3, including that a promotion between channels requires an explicit event.

M4. Projection and attribution

Implement the layout procedure with the projection key and the attribution diff, which reports whether a positional difference between two projections is due to the graph, the constraints, or the algorithm. Implement constraint events. Testable: invariant 4.

M5. Explanation and the text client

Implement the event-chain resolver used by every client, and build the text navigator as the first client. It displays the graph in a plain, link-following form, supports an explain action on every displayed relation, and keeps no history of its own beyond the user's path. Testable:

invariants 2 and 10. This milestone is the first at which a person can use the system, and it gives early evidence on whether explanation is usable before any spatial work.

M6. Repair

Implement dependency traversal over the declared relation types, scope declaration, graph-version recording, completeness status, the dependent status events, and follow-up repairs. Testable: invariants 6 and 7, including the planted out-of-scope dependent case.

M7. Spatial client

Build the spatial navigator over the projection data file. It must encode relevance, support status, and admissibility separately, render provisional and unverified events distinguishably, render doors from derived admissibility, and route every scan to the same explanation resolver as the text client. This milestone introduces no new invariant. It is the point at which a rendering error can violate an invariant already established below it, so the audit suite is run against what the spatial client actually displays and not only against the data it reads.

M8. Testbed and baselines

Build the planted ground-truth generator, the four baseline conditions, and the ablation flags. Because the flags were designed in from M0, each ablation is a switch that disables one invariant's enforcement and is otherwise identical to the full system. The deliverable is the set of conditions and ablated variants required by the experimental program.

3.1 Milestone and invariant map

Milestone	Delivers	Invariants testable
M0	Schema, canonical form, validator	3 (structural), 8
M1	Ledger, effective set, derived graph	1
M2	Status values, actors, provisional handling	5, 11
M3	Support, defeat, relevance, admissibility	3, 9
M4	Projection key, layout, attribution diff	4
M5	Explanation resolver, text client	2, 10
M6	Repair, scope, completeness	6, 7
M7	Spatial client with audited rendering	all, against displayed output
M8	Testbed, baselines, ablation flags	experimental program

4 Corpus import

A first corpus is needed from M1 onward, since an empty ledger tests nothing. The proposed source is an existing body of linked technical writing, imported by script. Explicit references between documents become `cites` relations with the citing text recorded in the basis. Any relation proposed by similarity enters as provisional with the retriever as actor. Any claim of provenance for earlier work, including dates and titles that have not been checked against

original manuscripts, enters with the status `unverified`. The import is repeatable and is itself recorded, so the corpus can be regenerated from its sources.

5 Relation to existing projects

The spatial client is the natural point of contact with the Cutaway Reader design, which already fixes typed, semantically directed passages, a path-history rear view, a scan and beacon vocabulary, and a text projection of the same graph. The plan treats those design decisions as requirements on M5 and M7. Whether the browser engine from icepick is reused or replaced is an open decision, with the consideration that it already renders a flattened snapshot as a navigable world and would need to be changed to read a projection file with separate encodings for the three relations. Federation, identity, and permissions, which belong to the collaboration layer, are deferred: the specification's `partial` completeness status provides the hook for them without requiring them to exist.

6 Open decisions

1. The name and home of the repository.
2. Whether the core stays in Python or is rewritten after the schema and tests stabilize.
3. The layout procedure, which must be deterministic given a seed and must expose its version.
4. The adjudication policy that moves an event from provisional to established, which the specification leaves to the deployment.
5. The default set D of dependency-bearing relation types, and whether it is per-deployment or per-repair.
6. How a ledger accommodates the removal of an event for legal or safety reasons without breaking the append-only property, for example through redaction events that preserve identifiers.
7. Actor identity and signing, deferred to the collaboration layer but affecting the event schema if added later.

7 Risks

The largest risk is that the audit suite passes against the data while a client displays something different, which is why M7 audits displayed output. A second risk is schema churn: changes to the event schema invalidate every downstream component, so M0 should be treated as provisional until M3 has exercised all event kinds. A third risk is that the spatial client absorbs effort disproportionate to its evidential value, since M5 may already show whether explanation is useful. If effort is limited, the spatial client is the component to defer, and the text client with the full ledger, repair, and audit still supports a meaningful version of the experimental program restricted to the conditions that do not require spatial navigation.